Homework Assignment A

This document contains the write-ups for the three challenges of **Assignment A**.

Problem ID	Lab Name	Captured Flag	Steps
Problem 1	one_byte	`fsuCTF{0h_n0_r3v3r5i813_3ncryp7i0n}`	- Decode the Base64 string to get the raw bytes - Identify XOR encryption - Use the known `fsuCTF{` prefix to recover the first 7 key bytes - Determine the key length is 8 using the closing `}` byte - Brute force the last key byte and decrypt
Problem 2	My Public Key	`fsuCTF{A_Big_N_Does_Not_Matter_If_Its_Factors_Are_Known}`	- Identify the known RSA values: `n`, `e`, `c` - Factor `n` using factordb.com to obtain `p` and `q` - Compute `phi`, then derive `d` as the modular inverse of `e` - Decrypt the ciphertext with `m = c^d mod n`
Problem 3	The Cooler Cipher	`fsuCTF{You_Know_What_ROT13_Aint_Cool_Enough_Have_ROT47}`	- Identify that the ciphertext includes special characters, ruling out ROT13 - Try ROT47 as the "cooler" version - it fails - Brute force all 94 possible ASCII shifts - Identify ROT52 as the correct shift

---

PROBLEM 1 - one_byte

In this challenge, we are given a single file called `ciphertext.txt` with the following content:

```
IQYwJgc1NlUvKitVDAF+E3QHcAxrQn46dBsmFyoDegx3Gzg=
```

1. Analysis

The first thing I noticed is the `=` at the end of the string. I looked this up and found that this is typical of **Base64** encoding, which is used to represent binary data as text. So the first step is to decode it to see the actual bytes.

I ran this command in the terminal:

```
echo "IQYwJgc1NlUvKitVDAF+E3QHcAxrQn46dBsmFyoDegx3Gzg=" | base64 -d | xxd
```

Output:

```
00000000: 2106 3026 0735 3655 2f2a 2b55 0c01 7e13  !.0&.56U/*+U..~.
00000010: 7407 700c 6b42 7e3a 741b 2617 2a03 7a0c  t.p.kB~:t.&.*.z.
00000020: 771b 38                                  w.8
```

So after decoding we have 35 bytes of data that don't look like readable text at all. At this point I started thinking it could be **XOR encryption**. XOR works by applying a bitwise operation between each byte of the plaintext and a key. If you XOR the ciphertext with the same key, you get the plaintext back.

I first tried a **single-byte XOR brute force** - that means trying all 256 possible key values (one byte) and checking if any of them produces readable text. I wrote this Python script:

```
data = bytes.fromhex("21063026073536552f2a2b550c017e137407700c6b427e3a741b26172a037a0c771b38")

for key in range(256):
    result = bytes([b ^ key for b in data])
    try:
        decoded = result.decode('ascii')
        if all(32 <= c <= 126 for c in result):
            print(f"Key 0x{key:02x} ({key:3d}): {decoded}")
    except:
        pass
```

but no single byte produced fully readable text. I also tried some other approaches like repeating-key XOR brute force and a small dictionary of common CTF keys, but none of them gave a clean result either.

The problem was that with only 35 bytes and no extra information, there are too many possibilities. Then I remembered that the flag should start with `fsuCTF{...}`, which is something we know for sure. This is called a **known plaintext attack**: if I know part of the plaintext, I can recover part of the key directly, because:

```
ciphertext XOR plaintext = key
```

2. Decipher

I XORed the first 7 bytes of the ciphertext with the known prefix `fsuCTF{`:

```
data = bytes.fromhex("21063026073536552f2a2b550c017e137407700c6b427e3a741b26172a037a0c771b38")

prefix = b'fsuCTF{'

key_fragment = bytes([data[i] ^ prefix[i] for i in range(len(prefix))])
print(f"Key fragment: {key_fragment}")
```

Output:

```
Key fragment: b'GuEeSsM'
```

So the first 7 bytes of the key are `GuEeSsM`.

I noticed that the last byte of the ciphertext is `0x38` (position 34). If the key has length 8, then position 34 maps to `key[34 % 8]` = `key[2]`, which is `E` = `0x45`.

```
0x38 XOR 0x45 = 0x7d = }
```

The `}` is exactly the closing bracket of the flag format. This strongly suggests the key length is 8.

Now I only need to find `key[7]`. I tried all 256 possible values and kept only the ones that produced fully printable text:

```
data = bytes.fromhex("21063026073536552f2a2b550c017e137407700c6b427e3a741b26172a037a0c771b38")

key_fragment = b'GuEeSsM'

for k7 in range(256):
    key = list(key_fragment) + [k7]
    result = bytes([data[i] ^ key[i % 8] for i in range(len(data))])
    if all(32 <= c <= 126 for c in result):
        try:
            decoded = result.decode('ascii')
            print(f"key[7] = 0x{k7:02x} ({chr(k7)}): {decoded}")
        except:
            pass
```

```
key[7] = 0x65 (e): fsuCTF{0h_n0_r3v3r5i813_3ncryp7i0n}
```

Among all the candidates, only `key[7] = 0x65` ('e') produced text that actually makes sense. It says "oh no reversible encryption" written in leet speak.

The full key is `GuEeSsMe`.

Flag: `fsuCTF{0h_n0_r3v3r5i813_3ncryp7i0n}`

PROBLEM 2 - My Public Key

In this challenge, I am given a Python file called `rsa.py`. The file shows the RSA encryption process and at the bottom, in a comment, it shows the output of a real execution. Most values are REDACTED, but some are left visible.

1. Analysis

Looking at the file, I can identify what I have and what I am missing:

Value	Status
`n`	Known
`e`	Known (65537)
`c`	Known
`p`, `q`	REDACTED
`phi`, `d`	REDACTED
`m`	REDACTED

The values I have are:

```
n =
65335782081095098629798810218809083092921492680027437007969295305300338493406945793884409870656
38511182899257588401176563544134568780495589980276573209831
e = 65537
c =
22319321024286428301142152052525910130424136097431274376355239391129907242490586447949989426314
35583827186541004896371367693520755048537406818767080482346
```

In RSA, `n` is the product of two large prime numbers `p` and `q`. If I can find `p` and `q`, I can reconstruct the rest of the values and decrypt the message.

I go to http://factordb.com, paste `n` in the search field and submit it. The result shows **FF** (Fully Factored), which means the website already has the factors in its database. This means whoever generated this challenge used primes that were already known or stored somewhere, which completely breaks the security.

The website shows the two factors but I cannot easily copy them in full from the interface. Instead, I use the factordb API from the terminal:

```
curl "http://factordb.com/api?
query=65335782081095098629798810218809083092921492680027437007969295305300338493406945793884409
87065638511182899257588401176563544134568780495589980276573209831"
```

Output:

```
{
    "id":1100000008710412137,
    "status":"FF",
    "factors":
    [
        ["579728997393016972467429882135417473526269735146647470166155616266887953153337",1],
        ["112700559011026778313623222438110524958206042791305955170078924032539414202863",1]
    ]
}
```

Now I have both prime factors:

```
p = 579728997393016972467429882135417473526269735146647470166155616266887953153337
q = 112700559011026778313623222438110524958206042791305955170078924032539414202863
```

2. Decipher

With `p` and `q` I can now calculate the remaining values and decrypt the message. In RSA:

- `phi = (p - 1) * (q - 1)`
- `d` is the modular inverse of `e` with respect to `phi`
- To decrypt: `m = c^d mod n`

I write a short Python script to do all of this:

```
p = 579728997393016972467429882135417473526269735146647470166155616266887953153337
q = 112700559011026778313623222438110524958206042791305955170078924032539414202863
```

```
n = p * q
phi = (p - 1) * (q - 1)

e = 65537
d = pow(e, -1, phi)

c =
22319321024286428301142152052525910130424136097431274376355239391129907242490586447949989426314
35583827186541004896371367693520755048537406818767080482346

m_int = pow(c, d, n)
m = m_int.to_bytes((m_int.bit_length() + 7) // 8, 'big')

print(f'phi: {phi}')
print(f'd: {d}')
print(f'm: {m}')
```

Output:

```
phi:
65335782081095098629798810218809083092921492680027437007969295305300338493405239059296906585900
78144972247605316090343547238163866593801104321048363691632
d:
58782963129677760587263027690340704876497595165862608842911661021739309991686322472685396034775
06870014550678240657201533780171418127727670097598237891777
m: b'fsuCTF{A_Big_N_Does_Not_Matter_If_Its_Factors_Are_Known}'
```

Flag: `fsuCTF{A_Big_N_Does_Not_Matter_If_Its_Factors_Are_Known}`

PROBLEM 3 - The Cooler Cipher

In this challenge we are given a text file called `ciphertext.txt` with the following content:

```
2?Am~pG%;A+u:;C+#4-@+|y~[]+k5:@+m;;8+o:;A34+r-B1+|y~^aI
```

1. Analysis

The first thing I noticed is that the ciphertext contains not only letters but also numbers and special characters like `~`, `|`, `+`, `@`, etc. This is unusual for a basic cipher like ROT13, which only works on letters.

The hint in the problem title says: "The Cipher: Rot13 / The Cooler Cipher: ???????". I searched for what the "cooler" version of ROT13 could be and I found **ROT47**. While ROT13 rotates 13 positions within the 26 letters of the alphabet, ROT47 rotates 47 positions within all printable ASCII characters (from `!` to `~`, which is 94 characters total).

2. Decipher

First attempt - ROT47

I wrote a small Python script to apply ROT47 to the ciphertext:

```
ct = '2?Am~pG%;A+u:;C+#4-@+|y~[]+k5:@+m;;8+o:;A34+r-B1+|y~^aI'
print(''.join(chr(33 + (ord(c) - 33 + 47) % 94) if 33 <= ord(c) <= 126 else c for c in ct))
```

Output:

```
anp>OAvTjpZFijrZRc\oZMJO,.Z<dioZ>jjgZ@ijpbcZC\q`ZMJO/2x
```

The result was not readable at all, so ROT47 was not the right answer. However, the approach seemed correct - the problem is about rotating characters in the ASCII range. I thought maybe the shift was just different, so I decided to try all 94 possible shifts.

Brute force - all shifts

I modified the script to try every possible shift from 1 to 93:

```
ct = '2?Am~pG%;A+u:;C+#4-@+|y~[]+k5:@+m;;8+o:;A34+r-B1+|y~^aI'
for shift in range(1, 94):
    result = ''.join(chr(33 + (ord(c) - 33 + shift) % 94) if 33 <= ord(c) <= 126 else c for c
in ct)
    print(f'ROT{shift:2d}: {result}')
```

I went through the output looking for a readable line or something with the `fsuCTF{...}` format. The line that caught my eye was **ROT52**:

```
ROT52: fsuCTF{You_Know_What_ROT13_Aint_Cool_Enough_Have_ROT47}
```

That is the flag. Interestingly, the message inside the flag says "have ROT47", which was my first guess - the problem was designed to mislead you into trying ROT47 first, but the actual cipher used was ROT52.

Flag: `fsuCTF{You_Know_What_ROT13_Aint_Cool_Enough_Have_ROT47}`